低空经济 · 可审计数据分析 Agent

面向低空物流场景的可审计分析 Agent 二次开发

口径固化、几何可复核性与模型规模消融

技术研究报告 / 研究生论文形式

项目：低空物流可审计分析 Agent 二次开发

全部数字为本地实测，可复现

2026 年 11 月

面向低空物流场景的可审计分析 Agent 二次开发：口径固化、几何可复核性与模型规模消融

目录

面向低空物流场景的可审计分析 Agent 二次开发：口径固化、几何可复核性与模型规模消融

- 摘要
- 1 引言
- 2 背景与基座系统
- 3 数据与方法
- 4 实验设计
- 5 实验结果
- 6 讨论
- 7 结论与未来工作
- 参考文献
- 附录 A 复现命令
- 附录 B 关键结果文件
- 附录 C 贡献边界

摘要

以大语言模型为核心的数据分析系统普遍面临两类可靠性问题：模型自行执行算术导致数值不可信，以及业务口径在自然语言到查询的转换过程中发生漂移。开源项目 `gfirik/agentic-analytics` 采用「模型不碰算术」的设计，所有数值由 DuckDB 执行 SQL 产生，并由独立验证器在报告发布前逐条重算。本文在该框架上完成面向低空物流场景的二次开发，研究问题为：**在不修改模型层与核心状态机的前提下，将数据层与工具层领域化，能否在保持可审计性的同时提升分析结果的完整性与数值可靠性；以及当引入真实语言模型后，该收益是否随模型规模增大而消失。**

研究接入 IEEE DataPort 深圳前海 truck-drone 基准实例的公开参数，以 AST 解析公开构建脚本中的常量并用标准库确定性复现数据，新增需求聚合、禁飞区冲突检测、高度剖面三个口径明确的分析工具，并按上游「函数事实必须注册镜像重算规则」的契约扩展验证器；几何判定结果被物化为 DuckDB 表，使原本仅理解 SQL 的验证器仍可复核。构建 24 条场景评测题与独立 SQL 答案键，在零模型调用的确定性回放下，基线通用 SQL 工具链与低空化版本数值正确率均为 24/24，而答案完整性由 0.399 提升至 0.990（+59.03 个百分点），可复核数字由 24 个增至 242 个（10.08×）。进一步引入本地 Qwen2.5-1.5B / 3B / 7B 三档模型，模型自行撰写 SQL 的正确率为 0/24、2/24、5/24，模型仅选择工具而由工具计算的稳定在 22/24、23/24、23/24：随模型增强差距缩小但未消失（仍为 18 题）。实验过程中还定位并修正了一类编排层缺陷——只剪枝不补全参数会使更遵守指令的模型表现反而下降（7B 首跑 10/24，修正后离线重放为 23/24）。本文报告全部负结果、数据局限与威胁效度，并给出完整可复现的实验链路。

关键词：可审计数据分析；低空经济；DuckDB；验证器；工具层；大语言模型评测；参数幻觉；可复现性

---

1 引言

1.1 问题背景

低空物流运营分析具有高时空维度、强口径依赖的特点。诸如「某片区 14:00—16:00 的配送需求峰值出现在哪个起降节点」「某轨迹与禁飞区的冲突次数是多少」「需求最高的时段是否与禁飞区活跃时段重叠」等问题，既涉及常规分组聚合，也涉及多边形包含判定、时间窗约束与高度分层等业务定义。若由分析人员或语言模型每次临时编写查询，计算口径分散在临时 SQL 中，数值难以追溯，且同一指标在不同提问者笔下容易产生不一致的定义。

与此同时，通用大语言模型直接生成 SQL 存在已被引用的可靠性风险：模型可能选错列、混淆时间字段与业务时钟、以求和代替计数，或生成语法正确但语义错误的查询。开源分析 Agent 框架由此提出「模型不碰算术」（the model never does arithmetic）的架构约束：模型负责理解与叙述，确定性代码负责编译查询、执行计算与复核数值。

1.2 研究问题

本文围绕三个递进的问题展开：

- **RQ1**：在只改动数据层与工具层的边界下，能否把通用可审计分析 Agent 领域化为低空物流分析 Agent，且新增能力仍被原验证器完整复核？
- **RQ2**：在公平基线（基线具备通用 SQL 能力、不被禁用任何题型）下，领域化带来的收益体现在哪些可量化指标上？
- **RQ3**：引入真实语言模型后，工具层的收益是否随模型规模增大而消失？若否，残差失效的来源是什么？

1.3 贡献

1. 提出一种**免登录、可哈希复现**的公开基准数据接入方法：从 IEEE DataPort 页面公开的构建脚本中以 AST 读取常量，用标准库复现其生成过程，明确区分公开参数与派生构造。
2. 给出**几何计算纳入 SQL 验证器**的改造路径：将点包含判定结果物化为关系表，并为三个新工具注册镜像重算规则；同时指出验证器仅保证一致性、不保证几何算法正确性，需以独立第二实现与 golden cases 补足。
3. 设计**公平且可审计的评测协议**：答案键由独立 SQL 设定，基线不被人为削弱；以结构化、非 LLM 的 rubric 量化答案完整性。
4. 完成**四轮递进实验**（确定性主实验、解析器迭代、本地 LLM 四臂消融、三档模型规模消融），并如实报告编排层缺陷、零冲突数据局限与未验证项。

---

2 背景与基座系统

2.1 基座架构

`gfirik/agentic-analytics`（Apache-2.0，v0.1，TypeScript / Node 22 / Bun / DuckDB）是一个自托管分析引擎。其流水线为固定状态机：

```
INGEST → PROFILE → METRICS_DRAFT → MEASURE → DETECT → DECOMPOSE
       → INDEX_DOCS → EXPLAIN → DECIDE → VERIFY → RENDER
```

其中 `MEASURE`、`DETECT`、`DECOMPOSE`、`VERIFY` 均为确定性阶段，不使用模型。作业工作区是一个 DuckDB 文件，包含 `facts`、`claims`、`verification`、`sources` 等表。系统对外提供 CLI、HTTP API、MCP Server 与 HTML 报告四种「门面」。

2.2 事实与验证器契约

`packages/schema/src/report.ts` 定义：

```
Computation = { kind: "sql" | "function", sql?, params?, fn?, inputs? }
Fact        = { id, label, value, unit, computation, citations, verified }
```

`packages/verify/src/recompute.ts` 的 V2 复核规则是本文最关键的架构约束。其源码注释明确指出：SQL 事实按存储参数重新执行；**函数事实由镜像注册表 `RULES[fn]` 从输入事实重算；查不到规则的函数无法被验证，这本身即被视为引擎缺陷。** `verify.ts` 在 V2 出现不一致时抛出 `VerifyError`，整个作业不发布报告（fail closed）。此外还有 V1/V3–V8 覆盖引用有效性、因果措辞、章节完整性等。

该契约的直接推论是：**在该框架中新增一个分析函数，必须同时为其注册一条独立的重算规则。**这一约束把「可审计性」从设计口号转化为可强制执行的接口义务，也是本文二次开发的最小且正确的落点。

2.3 相关工作定位

与通用 Text-to-SQL 或 BI 问答相比，本工作的差异不在查询生成能力，而在于：(a) 数字出厂前必须被独立重算；(b) 业务口径固化在工具而非提示词里；(c) 评测基线具备同等 SQL 能力，因而对比结果不来自能力禁用。与「模型自由调用工具」的 Agent 框架相比，本基座采用固定流水线而非自由工具选择，因此本文在补充实验中额外引入了一个规则式自然语言解析器与一个本地 LLM 路由臂，用于覆盖问答式交互。

---

3 数据与方法

3.1 数据来源与可复现接入

主数据集为 IEEE DataPort 记录 `10.21227/3gxb-mj63`（CC BY 4.0，2026-09-30）：深圳前海 truck-drone 动态调度基准实例，含 27 个 WGS-84 服务节点（1 depot / 3 vertiport / 23 customer）、2 个时变禁飞区多边形、雨季天气 Markov 链与时段拥堵表，5 个运营日、每日期望 52 单、07:00—19:00 时域。

获取约束：53.1 KB 主归档受 IEEE SAML 登录保护；页面给出的配套代码仓库 `IMUJason/robust-truck-drone-dispatch` 实测 HTTP 404。但页面另提供免登录的 `DATASET SCRIPTS` 归档（9,409 B），其中 `build_case.py`（24,126 B）自包含全部地理与随机参数：节点坐标、禁飞区顶点与激活窗口、泊松订单过程 λ=(0.09, 0.03, 0.14)、三个固定 seed、拥堵分段与天气转移矩阵。

本文方法：以 `ast.literal_eval` 直接读取该公开脚本中的常量（含多重赋值），不执行脚本、不复制粘贴硬编码；再用标准库复现脚本中明文写出的生成过程。day0 以归档 seed `2026092302` 精确复现 **52 单**，与 DataPort 摘要一致；day1–4 为 `--order-seed` 变体，按泊松过程自然波动为 39 / 55 / 48 / 39 单，合计 233 单。全部输出附 SHA-256 清单，重跑逐字节一致。

3.2 数据分层与诚实标注

层次	内容	性质
公开参数	27 个 WGS-84 坐标、2 个 NFZ 多边形及窗口、订单过程与 seed、时域、拥堵表、天气矩阵	真实发表参数
按过程生成	`orders.csv` 233 行（目的地、重量、服务时长、时间窗）	可复现抽样
派生构造	`trajectory_points.csv` 2097 点（最近 vertiport 起飞、9 点线性插值、sha256 选层 {45,60,75,90} m、爬升/巡航/降落剖面）	**明确标注的合成构造，非实测飞行**

口径差异亦被显式记录：原基准把坐标乘以绕行系数 1.724 得到「道路等效米」用于路径优化，本文做真实地理分析故保留 WGS-84 不缩放；临时禁飞区按 120 真实米换算而非在缩放框架中构造；天气事实表因依赖不可得的 `dagcd._sample_weather` 而不生成，评测题亦不涉及天气。

3.3 一个必须报告的数据质量发现

该实例的两个禁飞区与配送网络在**几何上零交集**：机场区经度 113.792–113.842，而全部 27 节点与 2097 轨迹点经度 ≥ 113.845；临时区 240 m 方形亦无轨迹穿过。激活窗口内虽有 1741 个轨迹点，但无一落在区内。因此「轨迹–禁飞区冲突次数」在本数据上**恒为 0**，对任何工具链都一样。

本文没有据此制造提升数字，而是：(a) 将该发现作为数据质量结论报告并固化为回归测试；(b) 另用**明确标注的合成探针轨迹**单独验证几何算法，探针从不混入业务评测题或业务结论。

3.4 三个低空分析工具及其口径

工具	输入与计算口径	输出
`drone_demand_aggregate`	时间窗内 `COUNT(DISTINCT order_id)`，归属到轨迹 `seq=0` 记录的起飞 vertiport；按 `(hour_bucket, origin_node_id)` 分组	`total`、`by_hour`、`by_node`、`peak` 及 argmax 分组
`no_fly_conflict_count`	命中 = WGS-84 点在 NFZ 多边形内（奇偶射线法）**且**时间戳落在激活窗口 `[from, to)`；1 次冲突事件 = 同一 `(轨迹, 区)` 内一段 seq 连续的命中	`events`、`points`、`minutes`、`by_zone`、`window_points`
`altitude_distribution`	**仅** `phase='cruise'` **点**计入，按 15 m 层分组	各层点数、主导层、占比、巡航点比例

每个输出数字对应一个 `Fact`。`sql` 事实按存储参数重跑；`function` 事实由新增的三条镜像规则重算：

fn	镜像规则
`lowalt.demand_peak.v1`	`max(输入分组计数)`，并校验报告中的 argmax 分组确为最大值所在组
`lowalt.nfz_conflict.v1`	`Σ` 物化表 `conflict_events` 上按区的 SQL 计数
`lowalt.altitude_profile.v1`	`max(层计数) / Σ × 100`

3.5 可审计性扩展与两层保障

几何判定在 Python 中执行，偏离了「所有计算由 DuckDB 执行」的原教旨。补偿设计为：把逐点命中与事件分段物化为 `conflict_point_hits`、`conflict_events` 两张 DuckDB 表，使原本只理解 SQL 的验证器仍能以 `SELECT COUNT(*)` 复核每一个发布值。

但必须指出：**V2 只保证「报告数字 = 重算结果」的一致性。**若几何算法本身写错，物化表随之错误，V2 仍会通过。因此本文补上第二层保障——独立第二实现（角度和 winding number，与射线法无共享代码）在 41×41=1681 个网格点上判定 100% 一致，外加分段规则、时间窗、未注册函数 fail-closed 与零交集回归等共 12 项单测。**一致性靠验证器，正确性靠独立实现与 golden cases，两者不可互相替代。**

3.6 自然语言入口与三态解析

补充实验需要一个问答式入口。v1 解析器匹配「话题名词」，在复合题上失效；v2 改为识别**所求量**（订单量 / 冲突次数 / 高度层）并打分，显式返回 `EXACT` / `AMBIGUOUS` / `UNRESOLVED` 三态：两类别得分接近时不硬猜、交给 planner；无所求量时拒绝猜测。该设计对齐上游「本地规则先解析、只有真正歧义才交给模型」的原则，并使「模型介入率」成为可测指标。

---

4 实验设计

4.1 评测集与答案键

24 条场景题分三层，各 8 条：纯数据聚合、空间与时空、复合推理。每题由**独立编写的 SQL** 在原始表上计算期望值，不读取任何工具输出，答案键固化在 `golden.lock.json` 并带漂移校验。此做法沿用上游 `evals/real` 的约定。

4.2 基线公平性

常见做法是拿不具备空间能力的原始工具链去比空间题，基线必输，所得「提升」缺乏信息量。本文反其道而行：

- **基线臂**：通用只读 SQL 工具 `sql_aggregate` 直接执行独立 SQL 答案键（空间题另给 SQL 包围盒近似），**从未被禁用任何题型**，每题都能给出数值答案；
- **低空化臂**：同一套通用工具 + 三个注册的低空工具。

因此两版在数值正确率上可以持平，差异只能来自口径固化、事实拆分、几何精度与答案结构。

4.3 指标定义

指标	定义
数值正确率	工具输出与独立 SQL 答案键在容差内一致的题目比例
答案完整性 (answer relevancy)	结构化、非 LLM 的 rubric：答案是否同时给出数值 / 所属维度 / 时间窗 / 计算口径定义 / 分组明细中的检查项，取满足比例
可复核数字数	本次运行发布并经 V2 逐条重算的 `Fact` 数量
V2 复核通过率	重算一致的事实比例
工具选择正确率	自然语言解析器选中的工具与标注一致的比例
步数 / 耗时	工具调用步数与墙钟时间

4.4 确定性环境与本地模型

主实验与解析器迭代为**零模型调用**的规则式回放，无随机性、无网络。补充实验使用 `llama.cpp` CPU 引擎与本地 GGUF 权重，服务仅监听 `127.0.0.1`，OpenAI 兼容端点，固定 `temperature=0 / seed=42 / top_p=1`。**不使用任何 API key，数据不出本机。** 模型包括 Qwen2.5-1.5B Q4_K_M、3B Q4_K_M、7B Q3_K_M；7B 采用 Q3 单文件是内存约束（约 4.5 GB 可用）下的选择。

四个实验臂共用同一 schema 与同一口径定义：

1. `llm_sql`：模型读 schema 与全部定义，自行写一条只读 SQL；
2. `llm_sql_focused`：**对 SQL 臂更有利的消融**，仅给该题相关定义，用于检验失败是否为长 prompt 伪影；
3. `llm_tools_typed`：模型选工具，参数仅做类型/范围校验（快照保留）；
4. `llm_tools`：模型选工具，参数经类型校验 + 问句约束校验，由工具计算、发布事实并运行 V2。

问句约束校验**只读问题文本**（是否提到 day、`HH:MM–HH:MM`、具体禁飞区），不读答案键、不读正确工具。

---

5 实验结果

5.1 主实验：确定性对比

指标	基线	低空化版	差值
数值正确率	24/24 (100%)	24/24 (100%)	0.0 pp
答案完整性	0.399	**0.990**	**+59.03 pp**
可复核数字	24	**242**	**10.08×**
V2 复核通过率	100%	100%	0
平均步数	1.208	2.208	+1.0
平均耗时	4.51 ms	25.20 ms	+20.7 ms

分层结果：

类别	n	数值正确率	答案完整性	可复核数字
纯数据聚合	8	8/8 → 8/8	0.3125 → 1.0000 (**+68.75 pp**)	8 → 132 (16.5×)
空间 / 时空	8	8/8 → 8/8	0.5521 → 1.0000 (**+44.79 pp**)	8 → 20 (2.5×)
复合推理	8	8/8 → 8/8	0.3333 → **0.9688** (**+63.55 pp**)	8 → 90 (11.25×)

结论：在公平基线下，二次开发并未提升数值正确率——一个能写 SQL 的通用工具本来就能算对。收益在于把每个数字变成可独立重算的事实、把口径固化，以及把答案从裸数值变成携带维度、时间窗、定义与明细的完整结论。

5.2 几何能力与合成探针

检查	精确多边形	bbox 近似
冲突事件数	**1**	2（高估 100%）
命中点数	**9**	18（高估 100%）

独立第二实现在 1681 个网格点上与射线法判定 100% 一致；分段规则、时间窗过滤、未注册函数 fail-closed、真实数据零交集回归均通过。

5.3 解析器迭代

v1 工具选择 19/24，5 例失效集中在复合题（A05、S07、C01、C03、C07）。根因不是关键词不足，而是匹配话题名词：「禁飞区活跃的窗口内共有多少配送订单」被"禁飞区"带去空间工具。v2 改为所求量判定并引入三态后达到 **24/24 全 EXACT**；8 条独立编写的改写题 8/8，真歧义题返回 `AMBIGUOUS`，无关题返回 `UNRESOLVED`；主实验三项指标完全不变，证明迭代未污染计算层。单测总数 8 → 12。

5.4 本地 LLM 四臂阶梯（1.5B）

臂	数值正确率	说明
LLM 自写 SQL（完整 prompt）	**0/24**	仅 7 条 SQL 可执行；错误含把时间窗当 `day` 过滤、`SUM(weight_kg)` 当订单量、`hour_bucket = 6` 类型错误
LLM 自写 SQL（聚焦 prompt 消融）	4/24 (16.7%)	长 prompt 遵循能力是部分原因，非全部
LLM 选工具 + 仅类型校验	7/24 (29.2%)	模型几乎每题幻觉出**合法**的 `day=4`，schema 校验拦不住语义幻觉
LLM 选工具 + 类型 & 问句约束校验	**22/24 (91.7%)**	答案完整性 0.955、可复核数字 215、工具选择 22/24
规则式工具层（无 LLM）	24/24 (100%)	主实验参照

剩余 2 例失效为 C01 / C03 工具选错，与 5.3 节 v1 解析器失效题完全重合，均属「禁飞区活跃的窗口内有多少订单」这类被主题名词带偏的复合措辞。

5.5 模型规模消融（RQ3）

模型	LLM 自写 SQL	聚焦 SQL	LLM 选工具	工具选择	答案完整性	耗时/题
1.5B Q4_K_M	0/24 (0%)	4/24	**22/24 (91.7%)**	22/24	0.955	4.3 s
3B Q4_K_M	2/24 (8.3%)	5/24	**23/24 (95.8%)**	23/24	0.976	8.3 s
7B Q3_K_M	**5/24 (20.8%)**	4/24	**23/24 (95.8%)**	23/24	0.948	17.1 s
规则式工具层	—	—	**24/24 (100%)**	24/24	0.990	0.005 s

SQL 臂随模型规模**单调改善**，阶梯部分变平；但工具臂稳定在 91.7%–95.8%，**绝对差距由 22 题缩至 18 题（91.7 pp → 75 pp）仍未追平**。三档工具臂在聚合类与空间类均为 8/8，残差集中于复合推理类。

5.6 编排层缺陷及其修正

7B **首跑**工具臂仅 10/24，工具选择却为 23/24。诊断发现：提示词中「不要编造时间窗」被更遵守指令的 7B 过度执行，导致 `params = {}`，工具退回默认全天，A02 / S04 / C02 等题答成全天值。这不是模型能力下降，而是**编排层只做剪枝、不做补全**造成的结构性缺陷：模型越守规矩，反而越吃亏。

修法：`normalize_tool_params()` 同时执行 prune（丢弃幻觉 day / zone / 时间窗）与 fill（从问句抽取 day、时间窗、禁飞区并补入参数），仍不读答案键与正确工具。由于 `temperature=0` 下输出确定，本文以 `replay_params.py` **离线重放已保存的原始模型输出**重新执行工具，不重新推理、不修改任何一个模型输出：

	重放前	重放后
1.5B 工具臂	22/24	**22/24**（无副作用）
7B 工具臂	10/24	**23/24**

原始快照保留在 `llm_*_prereplay.json`；3B 使用修正后脚本直接运行，三档口径一致。该发现可表述为一条工程结论：**编排层必须同时做参数剪枝与补全，否则「模型更听话」会被误读为「模型能力下降」。**

---

6 讨论

6.1 收益来源的归因

四轮实验把「工具层收益」拆成四个可指认的来源：(1) 计算口径固化，使模型无需生成 SQL；(2) 类型化参数契约；(3) 挡住合法但错误的语义幻觉；(4) 答案结构与逐数字事实化。0→4/24 说明长 prompt 是部分因素；4→7/24 说明仅口径固化不足；7→22/24 说明约束校验是关键；22→24/24 的残差则是意图分类问题，已由 v2 所求量解析器解决。

6.2 Trade-off

- **可审计性的代价是性能**：逐分组产出事实并重跑 SQL，聚合类平均耗时 3.85 ms → 42.79 ms（约 11×），事实数 8 → 132。数据规模增大时需改为批量或增量复核。
- **口径固化 vs 灵活性**：口径写进工具保证一致性，但新增口径需改代码；临时 SQL 更灵活。
- **几何用 Python 而非纯 SQL**：偏离「计算全归 DuckDB」，以物化表补回 SQL 可复核性，但验证器仍不能发现几何算法本身的错误，必须叠加独立第二实现。
- **确定性评测 vs 真实交互**：零模型回放可复现、无噪声，但无法反映模型工具选择的随机性；补充实验以 `temperature=0` 保留确定性，代价是没有置信区间。

6.3 威胁效度

1. **模型规模**：三档均为本地 1.5B–7B CPU 小模型，7B 采用 Q3_K_M 量化（内存约束下的选择），其 SQL 臂 5/24 是偏保守的下界。结论**不能推广到商用强模型或经过 SQL 微调的模型。**
2. **信息条件**：工具臂带编排层，SQL 臂没有；这正是工具层的定义，但也意味着这不是同等条件下的纯模型能力对比。
3. **约束校验来源**：问句约束校验是确定性规则，虽不读答案，却借用了规则式解析思路。
4. **单次确定性运行**：每题一次、`seed=42`，不报置信区间。
5. **数据局限**：禁飞区与网络零交集使冲突恒为 0；几何正确性仅在合成探针上验证。轨迹为派生构造而非实测飞行。
6. **未验证项**：上游 `bun install` 在当前网络下持续失败，TS 补丁未经 `bun run check` 编译验证；可运行、数字已实测的部分是 Python 数据层、工具层与评测层，它严格镜像上游 `Fact` / `RULES` / V2 契约。
7. **交互形态**：上游是固定流水线生成整份报告，本文的自然语言问答入口由评测 harness 提供，两者通过同一份事实契约对齐，而非改动上游 CLI。

6.4 对数据分析实践的启示

在结构化、口径明确的业务分析任务中，「让模型写查询」与「让模型选工具」的可靠性差距，主要不来自模型的 SQL 语法能力，而来自**业务约束是否被代码化**。把口径、参数合法性与分组维度固化为类型化契约，比反复优化提示词更有效；同时，任何「自动校验」都必须配套独立正确性验证，否则只校验了自洽性。

---

7 结论与未来工作

本文在不修改模型层与核心状态机的前提下，完成了可审计分析 Agent 的低空物流领域化：以公开参数确定性重建数据，新增三个口径明确的分析工具，按上游契约为验证器注册镜像重算规则，并将几何结果物化以支持 SQL 复核。在 24 题公平评测上，确定性主实验的数值正确率持平（24/24），而答案完整性提升 59.03 pp、可复核数字提升 10.08×；三档本地语言模型实验显示，模型自写 SQL 的能力随规模改善但工具层稳定在 91.7%–95.8%，收益未随模型增强而消失。

未来工作按优先级为：(1) 在内存/GPU 条件允许时以 7B Q4 或更强模型复测，检验阶梯是否继续变平；(2) 构造模糊提问分布以测量 `AMBIGUOUS` 态的命中率与模型介入效果；(3) 将逐事实重算改为增量复核以降低 11× 开销；(4) 在依赖可安装环境完成 TS 补丁的编译与集成测试；(5) 接入具备真实禁飞区交集的运营数据，以在业务题上而非仅在合成探针上验证冲突检测。

参考文献

1. gfirik. *agentic-analytics: An AI analyst you can audit*. GitHub, Apache-2.0, v0.1, 2026. `https://github.com/gfirik/agentic-analytics`
2. Cao, J. X. *Seed-Controlled Benchmark Instances for Dynamic Truck–Drone Delivery Dispatch (Shenzhen Qianhai Case)*. IEEE DataPort, 2026. DOI `10.21227/3gxb-mj63`, CC BY 4.0.
3. Qwen Team. *Qwen2.5 Technical Report* / Qwen2.5-Instruct-GGUF 模型仓库（ModelScope）.
4. ggml-org. *llama.cpp* (build b12110), MIT License. `https://github.com/ggml-org/llama.cpp`
5. DuckDB Foundation. *DuckDB* (Python 2.0.0), MIT License.
6. 本项目实测结果文件：`EVAL_REPORT.md`、`lowalt/eval/results/*.json`、`lowalt/data/SOURCES.md`。

附录 A 复现命令

```
# 环境：Python >= 3.10，pip install duckdb pyyaml
python lowalt/data/build_dataset.py --out lowalt/data/out
python -m unittest discover -s lowalt/tests -v
python lowalt/eval/run_eval.py

# 补充实验（需本地模型，见交接文档 HANDOFF.md）
python lowalt/eval/run_llm_eval.py --start-server --tag 1p5b --max-tokens 160 --timeout 240
python lowalt/eval/replay_params.py
python lowalt/eval/compare_models.py
```

附录 B 关键结果文件

文件	内容
`lowalt/eval/results/comparison.json`	主实验 24 题逐题双链路明细
`lowalt/eval/results/summary.json`	主实验聚合指标
`lowalt/eval/results/golden.lock.json`	独立 SQL 答案键与漂移校验
`lowalt/eval/results/llm_summary_<tag>.json`	各模型规模的三臂结果
`lowalt/eval/results/llm_*_prereplay.json`	编排层修正前的原始快照
`lowalt/eval/results/model_scaling.md`	三档模型规模对比表
`lowalt/data/out/manifest.json`	数据行号、参数与 SHA-256

附录 C 贡献边界

上游仓库仅改动 `packages/verify/src/recompute.ts` 两处（新增 import、规则查找回退）并新增 `packages/verify/src/lowaltitude-rules.ts`；模型层、core 状态机、agent、report 与既有规则均未修改。其余全部位于新增目录 `lowalt/`。

本文档由 tools/md2pdf.py 生成；全部数字来自仓库内实测结果文件，可复现。